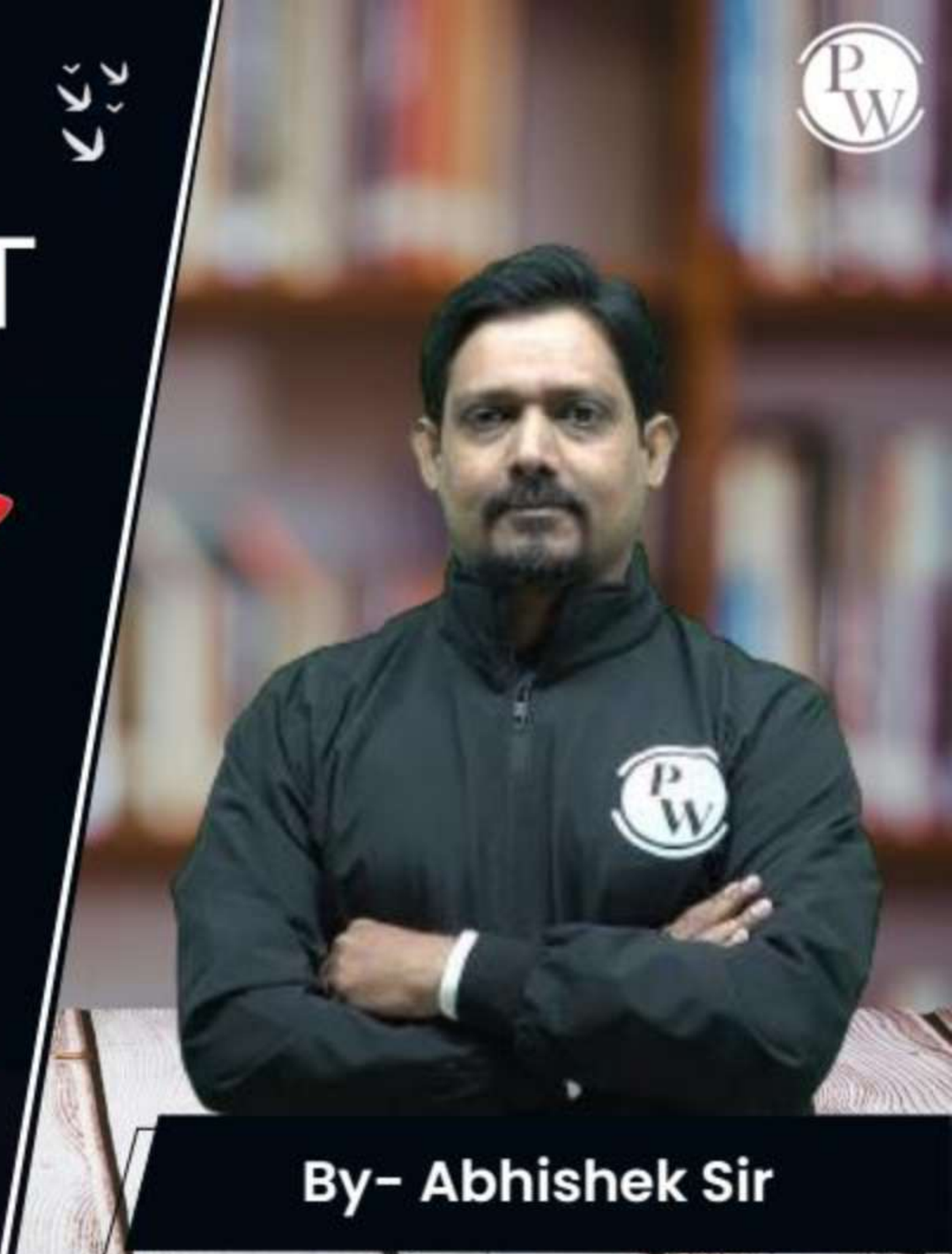
Computer Science & IT



C Programming

Function & Storage Class

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture



- Topic — function
- Topic — declaration definition
- Topic — control transfer
- Topic — Call by value
- Topic — Activation Record + Activation tree

Slide

Topics to be Covered



- Topic — Storage class
- Topic — auto
- Topic — static
- Topic — practice problem
- Topic

Slide



Function

Activation record

Storage class

- auto
- static
- register
- extern



Home Work

#Q.

```
#include <stdio.h>
int main()
{
int a,i;
a= 12.5/5+31.2/2*5-5;
for(i=a; i<90;i=i+3)
    {
      printf("%c\t", i+32);
    }
   return 0;
}
```

The output is-

(a) 107 110 113 116 119

(b) K L M N O

(c) k n q t w ✓

(d) Garbage values

$$\frac{89-75+1}{3} = \frac{15}{3} = 5$$

97 ✓ a = 2.5 + 15.6 * 5 - 5

98 ✓

99 ✓ a = 2.5 + 780 - 5

100 ✓

101 ✓ ✓ = 80.5 - 75.5 = a = 75

~~100~~ 102

103 ✓

104 ✓

105 ✓

106 ✓

107 ✓

75 + 32

107

Slide



Question

#Q.

Consider the following program:

```
#include <stdio.h>
int main()
{
    int i, j, n;
    for(i=1;i<=n;i++)
    for(j=1;j<=n;j++)
    printf("GATE Wallah");
    return 0;
}
```

How many times will the printf() statement be executed when n=32?

(a) 128 (b) 1024
(c) 512 (d) 256

$32 \times 32 = 2^{10} = 1024$

Slide



Question

Consider the following C program. Assume parameters to a function are evaluated from right to left.

```
#include <stdio.h>
int g(int p) { printf("%d", p); return p; }
int h(int q) { printf("%d", q); return q; }
void f(int x, int y) {
    g(x);
    h(y);
}
int main() {
    f(g(10),h(20));
}
```

Which one of the following options is the CORRECT output of the above C program?

(A) 20101020

(B) 10202010

(C) 20102010

(D) 10201020

20 10 10 20

10, 20

h

[A]

f(g(10), h(20));

f(10, 20) we need to evaluate the parameter

Slide

parameter evaluation

No order is defined in

C language



Question

#Q

Consider the following C program:

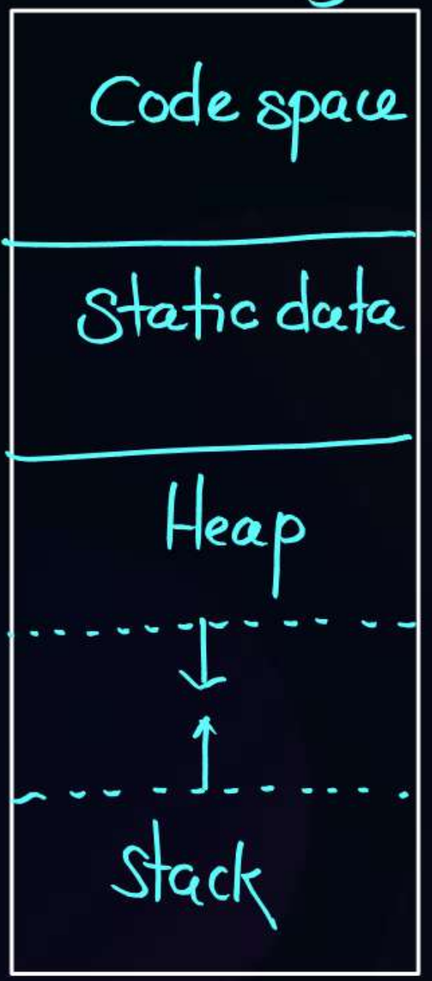
```
#include<stdio.h>
int f1(void);
int f2(void);
int f3(void);
int x=10;
int main()
{
    int x=1;
    x += f1() + f2 () + f3() + f2();
    printf("%d", x);
    return 0;
}
```

```
int f1() { int x = 25; x++; return x;}
int f2() {int x = 50; x++; return x;}
int f3() { x *= 10; return x;}
The output of the program is _____.
```



Storage Class

Storage class - Memory



RAM

Code space — C program - executable file

Loaded in code space.

Runtime stack :- Activation Record stored in stack.

Local variable, Return address

Register

Slide



Storage Class

static data



Storage Class

Heap



Storage Class

Storage class defines

1. The memory where variable will be stored.
2. Scope of variable i.e. the place where we can Access the variable.
3. Life time of variable, How long variable will be alive (allocate) & when deallocated.
4. Initial value of variable.

Slide



Storage Class

* auto : Local variable
* static
* register
* extern

Slide



Storage Class

auto is default storage class for local variable

auto - automatically

```
int fun() {
    int a;      // or   auto int a;
}
```

* runtime stack
* scope : within block
* Run time allocation
* Initial value garable
* as long as function is running variable is alive

Slide



Storage Class

global variable

a = 20

```
#include<stdio.h>
int a;
void fun() {

  printf("%d", a);

}
```

```
int main() {

  printf("%d", a);

  a = 20;

  fun();

  return 0;

}
```

I printf - 0

II printf - 20

Slide



Storage Class

```
#include<stdio.h>
int a; // static int a;
void fun() {

  printf("%d", a);

}
```

```
int main() {
  printf("%d", a);
  a=20;
  fun(); ✓
  return 0;
}
```

a = 20

I print – 0

II print - 20



Storage Class

Default storage class for global variable is static

* global variable stored in static data part of memory
* Any where in program;
* Compile time allocation
* As long as program is running
* global variable automatically initialize to 'zero'

Slide



Storage Class

* Local variable can also be declared as static

```
#include<stdio.h>
void fun() {
    static int a;
    a = a+10;
    printf("%d", a);
}
```

```
int main() {
fun();
fun();
return 0;
}
```

* GATE exam

* Scope - Local
* Lifetime : as long as program.

Slide



Storage Class

* Local variable can also be declared as static

```
#include<stdio.h>
void fun() {
    static int a;
    a = a+10;
    printf("%d", a);
}
```

```
int main() {
fun();
fun();
return 0;
}
```

a - still Local variable

a - Initial value = 0

print - 10

Slide



Storage Class

a [10] ←

* Local variable can also be declared as static

```
#include<stdio.h>
void fun() {
    static int a=0;
    a= a+10;
    printf("%d", a);
}
```

```
int main() {
fun();
fun();
return 0;
}
```

second time

It will print 20

10, 20

Slide



Storage Class

a [10] ←

* Local variable can also be declared as static

```
#include<stdio.h>
void fun() {
        int a=0;✓
        a= a+10;
        printf("%d", a);
}
```

```
int main() {
fun();        10 ✓
fun();        10 ✓
return 0;
}
```

Slide



Storage Class

a [10] ←

* Local variable can also be declared as static

* Local static initialize automatically to zero

* Local static variable Initialized only once.

* Local static variable preserve the value during multiple function call because it maintains a single copy.

Slide



Question

Consider the following C program:

```
#include <stdio.h>
int jumble(int x, int y){
  x=2*x+y;
  return x;
}
int main(){
  int x=2, y=5;
  y= jumble(y,x);
  x= jumble(y,x);
  printf("%d \n", x);
  return 0;
}
```

The value printed by the program is _____

X=y , Y=x

X=5 Y=2

x = 10+2 = 12

X 2

y ~~5~~ 12

y = jumbl(y, x) . 12

Slide



Question

Consider the following C program:

```
#include <stdio.h>
int jumble(int x, int y){
  x=2*x+y;
  return x;
}
int main(){
  int x=2, y=5;
  y= jumble(y,x);
  x= jumble(y,x);
  printf("%d \n", x);
  return 0;
}
```

The value printed by the program is [26]

x=y y=x

x=12 y=2

x=2x12+2

x=26

x: ~~2~~26 y: ~~5~~12

x=jumble(y,x) x=26

Slide



GATE 2017

#Q

Consider the C program fragment below which is meant to divide `x` by `y` using repeated subtractions. The variables `x`, `y`, `q` and `r` are all `unsigned int`.

```
while (r >= y)    {
r = r - y;
q = q +1;
}
```

Which of the following conditions on the variables `x`, `y`, `q` and `r` before the execution of the fragment will ensure that the loop terminates in a state satisfying the condition `x== (y*q + r)`?

(A) `(q==r) && (r==0)`

(B) `(x>0) && (r==x) && (y>0)`

(C) `(q==0) && (r==x) && (y>0)`

(D) `(q==0) && (y>0)`

$\frac{7}{2}$ Dividend Q $y>0$

divisor R

Remainder = Divided = 7 $Q=0$

$7-2=5$ Remainder, $Q+1=1$

$5-2=3$ Remainder $= Q=2$

$3-2=1$ Remainder, $Q=3$

Slide



Question

#Q

Consider the following C program:

```
#include<stdio.h>
int f1(void);
int f2(void);
int f3(void);
int f4(int, int, int, int);
int x=10;
int main(){
    int x=1;
    x+= f4(f1(), f2() , f3() , f2());
    printf("%d", x);
    return 0;
}
```

```
int f1() { int x = 25; x++; return x;}
int f2() {static int x = 50; x++; return x;}
int f3() { x *= 10; return x;}
int f4(int a, int b , int c , int d) {return
a+b+c+d;}
```

26 51 100 52

X = 1+26+51+100+52

(A) 228

(B) 229

(C) 230

(D) 231

The output of the program is _____.

Slide



2 mins Summary

Topic storage class

Topic auto

Topic static

Topic

Topic

THANK - YOU

